

《推荐技术算法与实现》 期末大作业技术报告

一、小组基本信息

1. 小组名称

汪汪队gogogo

2. 小组成员列表

序号	姓名	学号	分工说明
1	代乐	71285901045	数据清洗,数据增强,特征处理
2	施俊杰	71285901029	训练策略,参数设置,优化

二、排名截图

✖ Team Performance		View Leaderboard	
 Industrial Track	Rank	Best Score	Best Score Submission Time
	372	0.84417	2026-05-16 20:47:03

 Algorithm Explorers957 Industrial Track 汪汪队gogogo Upload Resume 			
Team Performance View Leaderboard			
 Industrial Track	Rank	Best Score	Best Score Submission Time
	372	0.84417	2026-05-16 20:47:03

三、使用的模型和训练方法简介

1. 使用的模型

本项目采用 PCVRHyFormer（Post-Click Conversion Rate Hybrid Transformer）模型，用于广告点击后转化率（pCVR）预测任务。该模型是一种面向多序列推荐场景的混合 Transformer 架构，核心思想是将用户/物品的静态特征与多域行为序列统一编码，通过交叉注意力机制融合多源信息，最终输出二分类预测。

模型整体结构如下：

(1) 输入层

模型接收五类输入特征：

- 用户整数特征（31 个特征）：包含用户画像的离散属性，经 Embedding 表映射为稠密向量；
- 物品整数特征（14 个特征）：包含广告/物品的离散属性；
- 用户稠密特征（10 个特征，总维度 918）：用户侧的连续型数值特征；
- 序列特征（4 个行为域）：分别来自 seq_a（9 个特征）、seq_b（14 个特征）、seq_c（12 个特征）、seq_d（10 个特征）四个行为域，记录用户在不同场景下的历

史交互序列；

- 时间桶特征：对序列中每条行为的时间差进行分桶编码（共 65 个桶），捕捉时间衰减信息。

(2) Non-Sequence (NS) Tokenizer

静态特征通过 RankMixerNSTokenizer 转化为固定数量的 NS Token。该模块将所有特征的 Embedding 拼接后均匀切分为若干 chunk，每个 chunk 经线性投影生成一个 NS Token。当前配置生成 5 个用户 NS Token + 1 个用户稠密 Token + 2 个物品 NS Token = 共 8 个 NS Token。

(3) 序列编码器 (Sequence Encoder)

每个行为域的序列独立经过一层 Transformer Encoder 进行编码。编码器采用 Pre-LayerNorm 架构，包含多头自注意力（4 头）和前馈网络（SwiGLU 激活，扩展倍数 4），隐藏维度 64。模型还支持 SwiGLU Encoder 和 LongerEncoder（Top-K 压缩编码器）两种备选方案。

(4) 查询生成器 (Query Generator)

MultiSeqQueryGenerator 模块为每个行为域独立生成 2 个 Query Token。它将序列的均值池化向量与 NS Token 拼接，经线性变换后产生查询向量，用于后续从序列中提取关键信息。4 个域共生成 8 个 Query Token。

(5) HyFormer 融合块

核心融合模块 MultiSeqHyFormerBlock 堆叠 2 层，每层包括：

- Sequence Evolution：对每个域的序列进行进一步编码；
- Query Decoding：Query Token 通过交叉注意力从对应域的序列中抽取信息；
- Token Fusion：将所有 Query Token 和 NS Token 拼接（共 16 个 Token）；
- Query Boosting：通过 RankMixerBlock 进行 Token 间的无参数重排混合和逐 Token FFN 变换。

(6) 输出层

将所有 Token 展平后经线性投影到 d_model 维度，再通过 3 层 MLP 分类器输出最终预测值 (Logit) 。

模型关键超参数：

参数	值
隐藏维度 d_model	64
嵌入维度 emb_dim	64
HyFormer 层数	2
注意力头数	4
FFN 扩展倍数	4
Query Token 数/域	2
总参数量	160,955,137
稀疏参数 (Embedding)	158,453,312
稠密参数	2,501,825

2. 数据处理方法

数据来源：训练数据以 Parquet 列式存储格式提供，包含用户特征、物品特征、多域行为序列及标签。

数据加载方式：使用 PyTorch 的 IterableDataset 进行流式加载，避免一次性将全部数据载入内存。数据按 Row Group 划分，90% 用于训练，10% 用于验证。

特征处理方法：

- 离散特征处理：
 - 空值和非法值 (≤ 0) 统一映射为 0 (作为 padding/unknown 标记) ；
 - 超出词表大小的值裁剪到合法范围；
 - 多值特征 (如用户兴趣标签) 进行变长填充，统一到固定槽位数。
- 稠密特征处理：

- 直接读取为浮点数组，不足最大维度的部分用 0 填充。

3. 序列特征处理：

- 每个行为域包含多个特征维度，统一处理为 (batch, num_features, max_len) 的三维张量；
- 序列最大长度：seq_a/seq_b 为 256，seq_c/seq_d 为 512；
- 对于高基数特征（词表 > 100 万），跳过 Embedding 构建以节省内存。

4. 时间桶编码：

- 计算序列中每条行为与当前时间的时间差；
- 使用 64 个边界值（从 5 秒到约 1 年）将时间差分桶，生成桶 ID 作为额外特征。

数据加载优化：

- 预分配 NumPy 缓冲区，避免每批次重复分配内存；
- 融合填充循环（Fused Padding），单次遍历直接写入三维缓冲区；
- 预计算列名到索引的映射，避免逐行字符串查找；
- 使用 file_system 共享策略替代默认的 /dev/shm 共享内存，防止多 worker 耗尽共享内存。

3. 训练方法

训练框架：基于 PyTorch 实现，核心训练逻辑封装在 `PCVRHyFormerRankingTrainer` 中。

双优化器策略：

- AdamW（用于稠密参数）：学习率 $1e-4$ ，动量参数 $\text{beta} = (0.9, 0.98)$ ；
- Adagrad（用于稀疏 Embedding 参数）：学习率 0.05，无权重衰减。

Adagrad 更适合处理高维稀疏更新的 Embedding 参数，而 AdamW 对稠密的 Transformer 参数具有更好的收敛性。

学习率调度：

- Cosine Annealing 调度策略，配合线性 Warmup；

- Warmup 比例为总训练步数的 5%，学习率从 0.1 倍线性升至 1.0 倍；
- 之后按余弦曲线从 1.0 倍衰减至 0。

损失函数：

- 使用 BCEWithLogitsLoss（带 Logits 的二元交叉熵损失）；
- 应用 Label Smoothing（平滑系数 0.01）：
$$\text{label_smooth} = \text{label} * 0.99 + 0.5 * 0.01$$
，缓解标签噪声；
- 备选方案：Sigmoid Focal Loss（alpha=0.1, gamma=2.0），未启用。

训练配置：

参数	值
批大小 (batch_size)	256
最大轮数 (num_epochs)	999
早停耐心值 (patience)	5
随机种子 (seed)	42
梯度裁剪最大范数	1.0
DataLoader workers	8
Shuffle buffer	20 batches

训练流程：

每轮（Epoch）依次执行：前向传播 -> 计算 BCE 损失 -> 反向传播 -> 梯度裁剪（max_norm=1.0） -> 优化器更新 -> 调度器步进。每轮结束后在验证集上计算 AUC 和 LogLoss。若验证 AUC 连续 5 轮未提升则触发早停。

实际训练结果：

轮次	训练损失	验证 AUC	验证 LogLoss
1	0.4466	0.9766	0.3136
2	0.4071	0.9845	0.3052
7	0.4076	0.9853	0.3037
12	0.3820	0.9790	0.2891

模型在第 7 轮达到最佳验证 AUC (0.9853) ，第 12 轮触发早停，总训练时间约 15 分钟。最终保存的最佳模型检查点为 `global_step28` 。

4. 模型优化与改进

在实验过程中，我们进行了以下优化与改进：

(1) 双优化器分离策略

将模型参数分为稀疏参数（Embedding 表，约 1.58 亿参数）和稠密参数（Transformer 层，约 250 万参数），分别使用 Adagrad 和 AdamW 优化器。这是因为 Embedding 参数更新极为稀疏，Adagrad 的自适应学习率机制能更好地处理低频特征。

(2) Embedding 冷重启策略

从第 1 轮（epoch 1）开始，每轮训练结束后对所有高基数 Embedding 表进行重新初始化（Re-initialization），并重建 Adagrad 优化器。该策略参考快手 MultiEpoch 技术，目的是防止 Embedding 过拟合，使模型在每轮都能从新的初始化出发学习更鲁棒的表示。实验中共 95 个高基数 Embedding 被重新初始化，仅保留 1 个低基数 Embedding 的状态。

(3) RankMixer Token 混合机制

在 HyFormer 融合块中，采用 RankMixerBlock 进行 Token 间信息交互。该模块通过无参数的 reshape/transpose 操作实现 Token 重排（Token Rewiring），再经逐 Token FFN 变换，以极低的计算开销实现跨 Token 的信息融合。

(4) 混合精度训练（AMP）

启用 PyTorch 自动混合精度训练（float16 前向/反向 + float32 损失缩放），配合 GradScaler 防止梯度下溢。在 GPU 环境下可显著加速训练。

(5) 高基数特征跳过与额外正则化

- 词表大小超过 100 万的序列特征跳过 Embedding 构建（`emb_skip_threshold=1000000`），减少内存占用；
- 词表大小超过 1 万的 ID 类序列特征施加 2 倍 Dropout（`seq_id_threshold=10000`），抑制高基数 ID 特征的过拟合。

(6) NS Tokenizer 选型

对比了 GroupNSTokenizer（按语义分组，均值池化后投影）和 RankMixerNSTokenizer（拼接切分后投影）两种方案，最终选用 RankMixer 变体，因其 Token 数量可灵活调控且信息保留更完整。

(7) 学习率 Warmup + 余弦退火

训练前期使用线性 Warmup 避免初始阶段学习率过大导致发散，后期使用余弦退火平滑降低学习率，有助于模型跳出局部最优。

(8) 数据加载性能优化

通过预分配缓冲区、融合填充循环、列索引预计算等手段优化数据加载管线，减少 CPU 瓶颈对训练速度的影响。